National College of Ireland

Cloud Platform Programming

NoSQL Services

Outline

- Introduction to NoSQL
- Database-as-a-Service – NoSQL Services
- Amazon DynamoDB
  - Partitions and Data Distribution
  - Read/Write Throughput
  - Streams
  - Global Tables
  - Core Operations

Outline

- Introduction to NoSQL
- Database-as-a-Service – NoSQL Services
- Amazon DynamoDB
  - Partitions and Data Distribution
  - Read/Write Throughput
  - Streams
  - Global Tables
  - Core Operations

Storing Data

- Most systems rely on some sort of data storage
  - Persistence
- Storing and retrieving data

Storing Data

- Most systems rely on some sort of data storage
  - Persistence
- Storing and retrieving data
- Where would you store your data?

Storing Data

- Most systems rely on some sort of data storage
  - Persistence
- Storing and retrieving data
- Where would you store your data?
  - Data can be stored in flat files
  - More complex data needs more complex structures
    - The database

Relational Database

Branch

branchNo	street	city	postcode
B005	22 Deer Rd	London	SW1 4EH
B007	16 Argyll St	Aberdeen	AB2 3SU
B003	163 Main St	Glasgow	G11 9QX
B004	32 Manse Rd	Bristol	BS99 1NZ
B002	56 Clover Dr	London	NW10 6EU

- A relational database (RDB) is based on the relation model proposed by Edgar Codd in 1970
- A collection of data items with predefined relationships between them
- RDBMS are made up of tables
  - Each column represents an attribute
  - Each row represents a collection of related values of an entity
- Each table typically has a *primary key*
- Tables are linked via *foreign keys*

Staff

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000	B005

source: Connolly & Begg, *Database Systems: A Practical Approach to Design, Implementation, and Management*

Relational Database

- To retrieve the data from a database we have to use a query
- Structured Query Language (SQL)
  - Based on relational algebra and tuple relational calculus

Relational Database

- Complex data structures can be mapped to multiple tables
  - Joins
- *Normalization* is the process of organizing a RDB according to some properties (normal forms) to reduce data redundancy and improve data integrity

Relational Database

- Drawbacks
  - Rigid data model
  - Speed
  - Scalability

NoSQL Databases

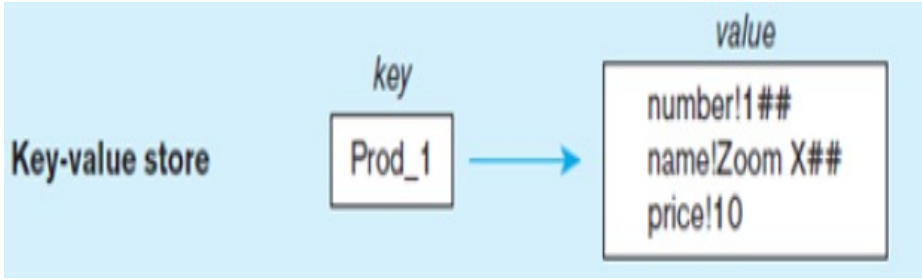
- Stands for **N**ot **O**nly **SQL**
- Class of non-relational data storage systems
- Usually do not require a fixed table schema nor do they use the concept of joins

NoSQL Databases Benefits

- Flexible
- Fast
- Scalable

NoSQL Databases Types

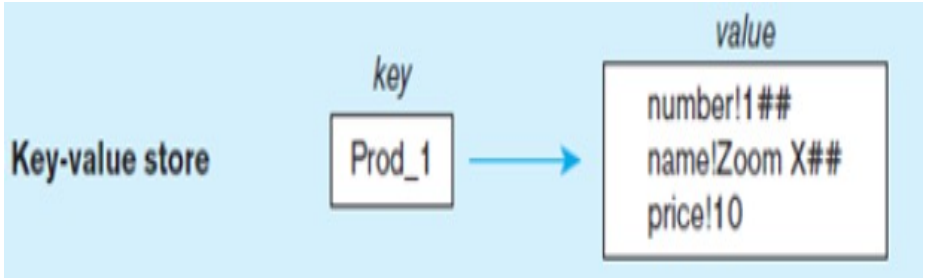
- Key-Value DB
  - A collection of key-value pairs
  - The key is a unique identifier
  - Database has no knowledge of the structure or meaning of the values



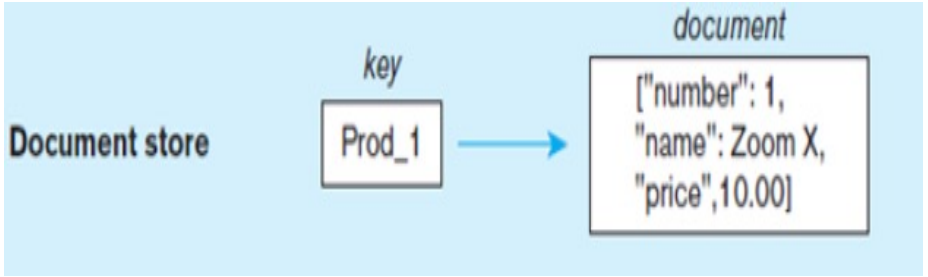
source: Connolly & Begg, *Database Systems: A Practical Approach to Design, Implementation, and Management*

NoSQL Databases Types

- Key-Value DB
  - A collection of key-value pairs
  - The key is a unique identifier
  - Database has no knowledge of the structure or meaning of the values

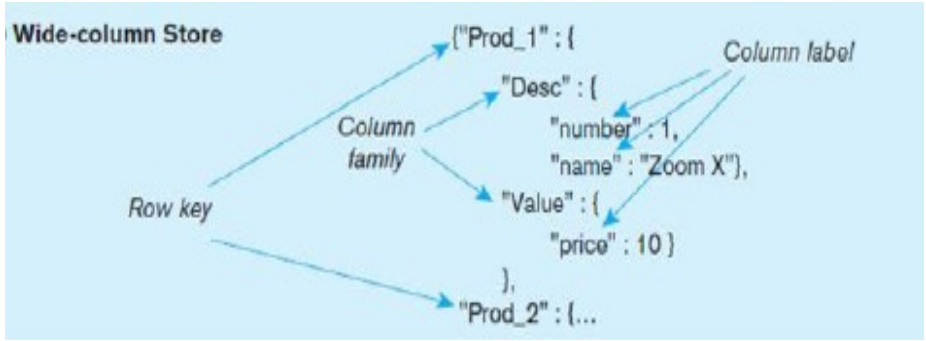


- Document DB
  - A subclass of the key-value DB
  - Document is structured so specific elements can be manipulated separately



source: Connolly & Begg, *Database Systems: A Practical Approach to Design, Implementation, and Management*

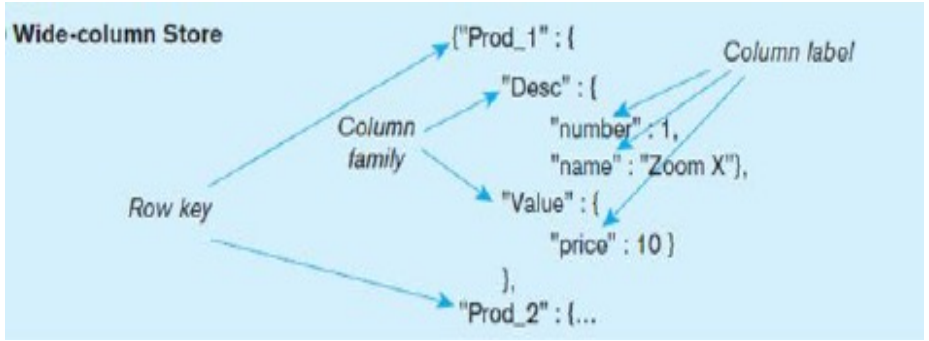
NoSQL Databases Types



- Column-based (wide-column) DB
  - Rows and columns
  - Distribution of data based on both key values (rows) and columns

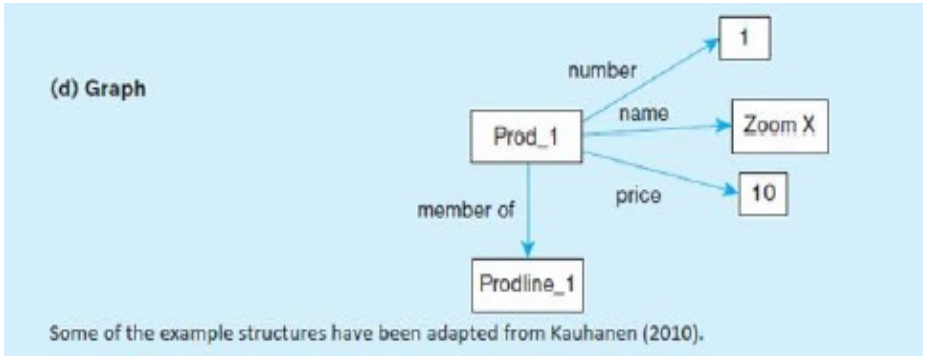
source: Connolly & Begg, *Database Systems: A Practical Approach to Design, Implementation, and Management*

NoSQL Databases Types

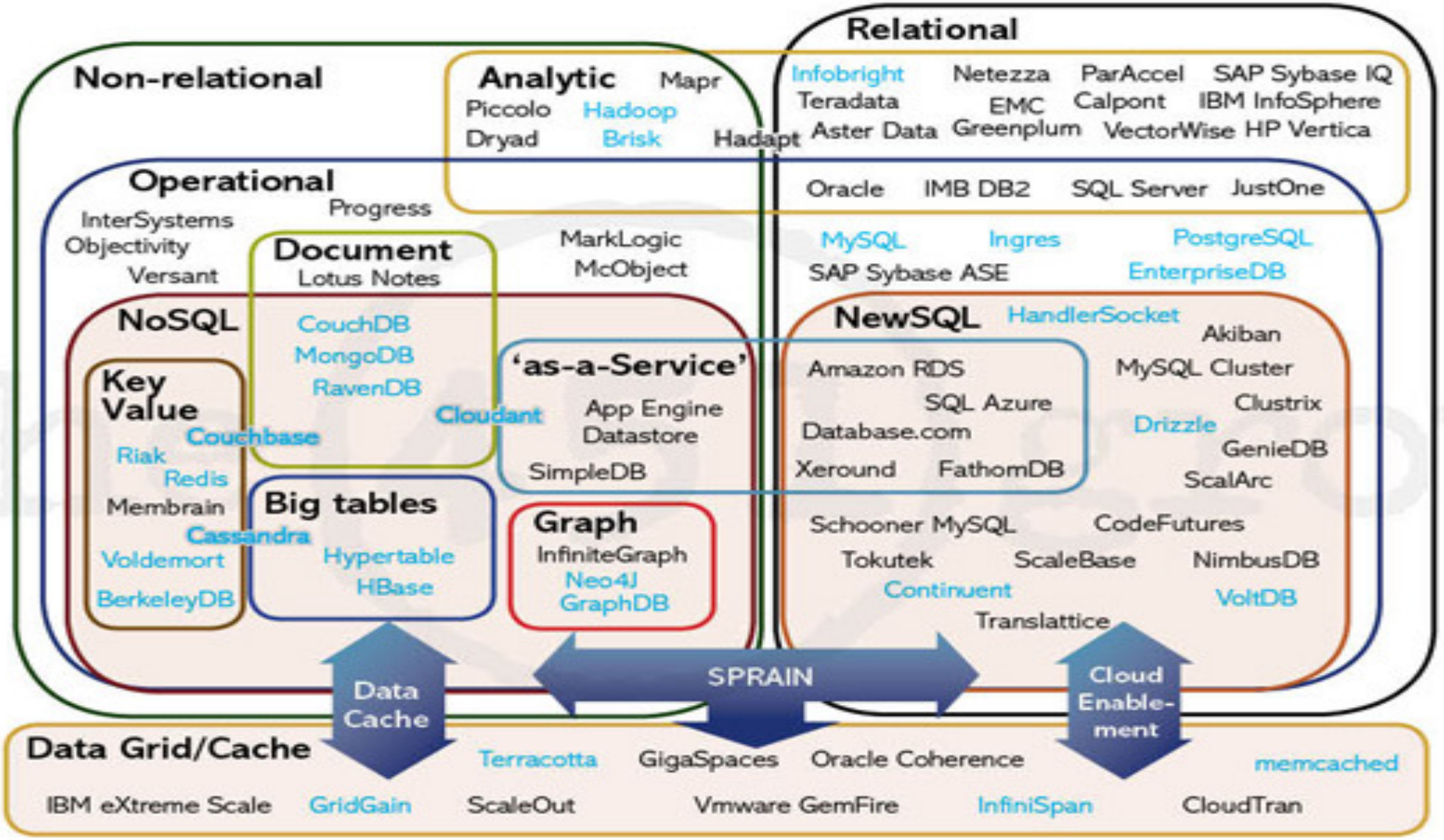


- Column-based (wide-column) DB
  - Rows and columns
  - Distribution of data based on both key values (rows) and columns

- Graph-oriented DB
  - Maintain information regarding the relationships between data items
  - Nodes with properties
  - Edges between nodes (relationships)
  - Edges can also have properties



source: Connolly & Begg, *Database Systems: A Practical Approach to Design, Implementation, and Management*



source: 451 Group (2011)

NoSQL Scenarios

- Mobile apps
- Personalization
- Real-time analytics
- IoT apps
- Content management

DBaaS – NoSQL Services

- Database-as-a-Service (DBaaS) is a cloud computing service that enables users to access and use a cloud database
- Cloud Service Provider (CSP) responsible to maintain and manage the DB, and ensure that the DB system is available and secure

DB-as-a-Service – NoSQL Services

Examples

- Amazon DynamoDB
- Azure Cosmos DB
- IBM Cloudant
- Google Cloud Firestore

Outline

- Introduction to NoSQL
- Database-as-a-Service – NoSQL Services
- Amazon DynamoDB
  - Partitions and Data Distribution
  - Read/Write Throughput
  - Streams
  - Global Tables
  - Core Operations

Amazon DynamoDB

- A key-value and document database
- Fully managed
- Performance: single-digit millisecond performance at any scale
- High Availability and Durability

source: https://aws.amazon.com/dynamodb

Amazon DynamoDB – Use Cases

- Ad Tech
  - e.g. user profiles, clicks, and visited links
- Gaming
  - e.g. game state, player data, leaderboards
- Retail
  - e.g. shopping carts, inventory tracking, customer profiles
- Banking and Finance
  - e.g. user transactions, event-driven transaction processing
- Media and entertainment
  - e.g. media metadata stores
- Software and internet

source: https://aws.amazon.com/dynamodb

Amazon DynamoDB – Terminology

- DynamoDB stores data in tables
- A **table** is a collection of data
- **Items** – each table contains zero or more items
  - An *item* is a group of attributes that is uniquely identifiable among all of the other items
- **Attributes** – each item is composed of one or more attributes
  - An *attribute* is a fundamental data element

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.CoreComponents.html

Amazon DynamoDB – Terminology

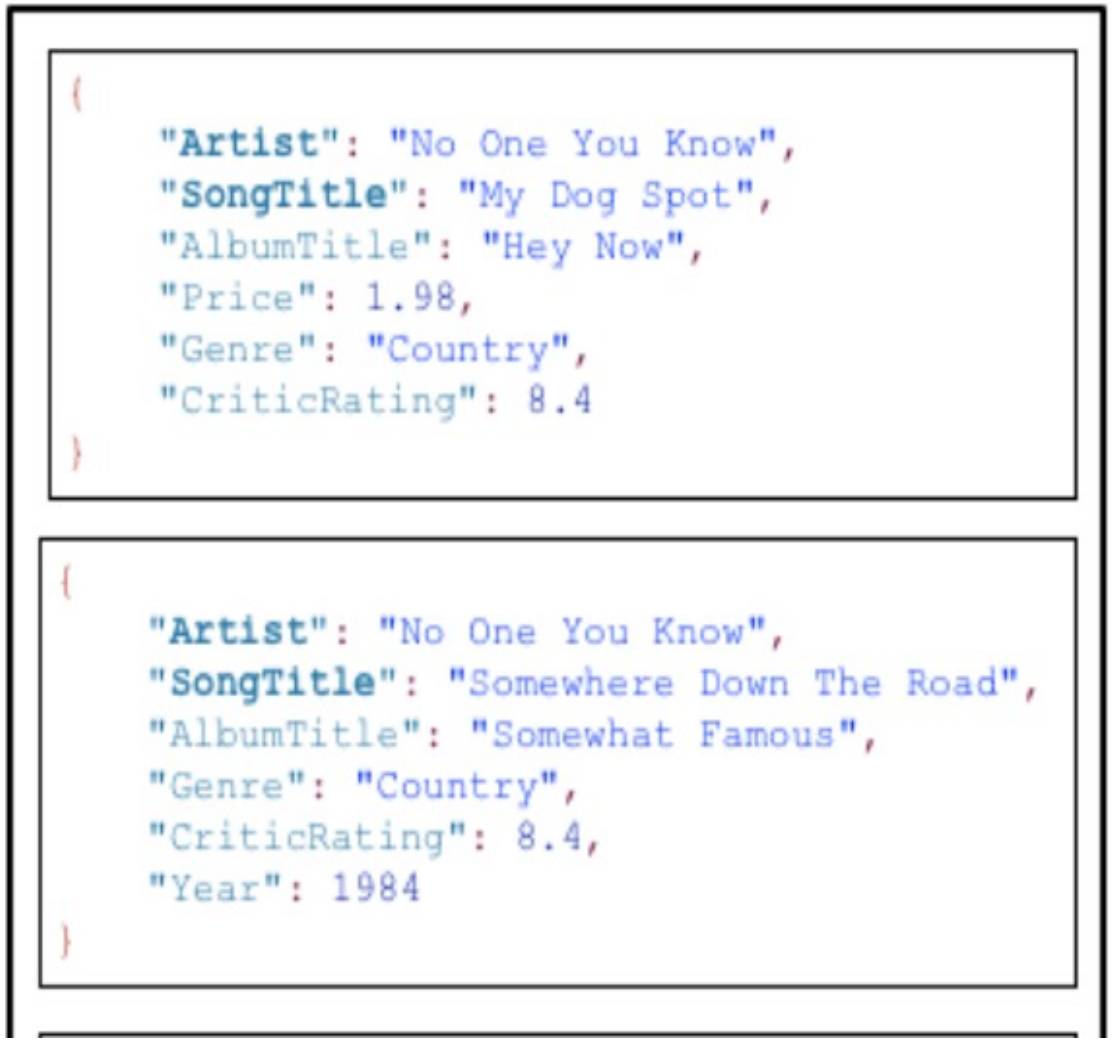
- ***Primary key*** – A table has a primary key that uniquely identifies each item in the table
- ***Simple primary key*** - composed of one attribute known as the *partition key*
  - *e.g. PersonID*



source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.CoreComponents.html

Amazon DynamoDB – Terminology

- ***Primary key*** – A table has a primary key that uniquely identifies each item in the table
- ***Composite primary key*** - composed of two attributes
  - the *partition key*
    - e.g. Artist
  - the *sort key*
    - e.g. SongTitle



source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.CoreComponents.html

Amazon DynamoDB – Items and Attribute Types

- DynamoDB is not constrained by a pre-defined schema
- An item can have any number of attributes with different data types
- Each attribute has a name and a value
- An attribute value can be one of the following data types:
  - Scalar types – Number, String, Binary, Boolean, and Null
  - Multi-valued types – String Set, Number Set, and Binary Set
  - Document types – List and Map

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.NamingRulesDataTypes.html#HowItWorks.DataTypes

Example: Amazon DynamoDB – Terminology



source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.CoreComponents.html

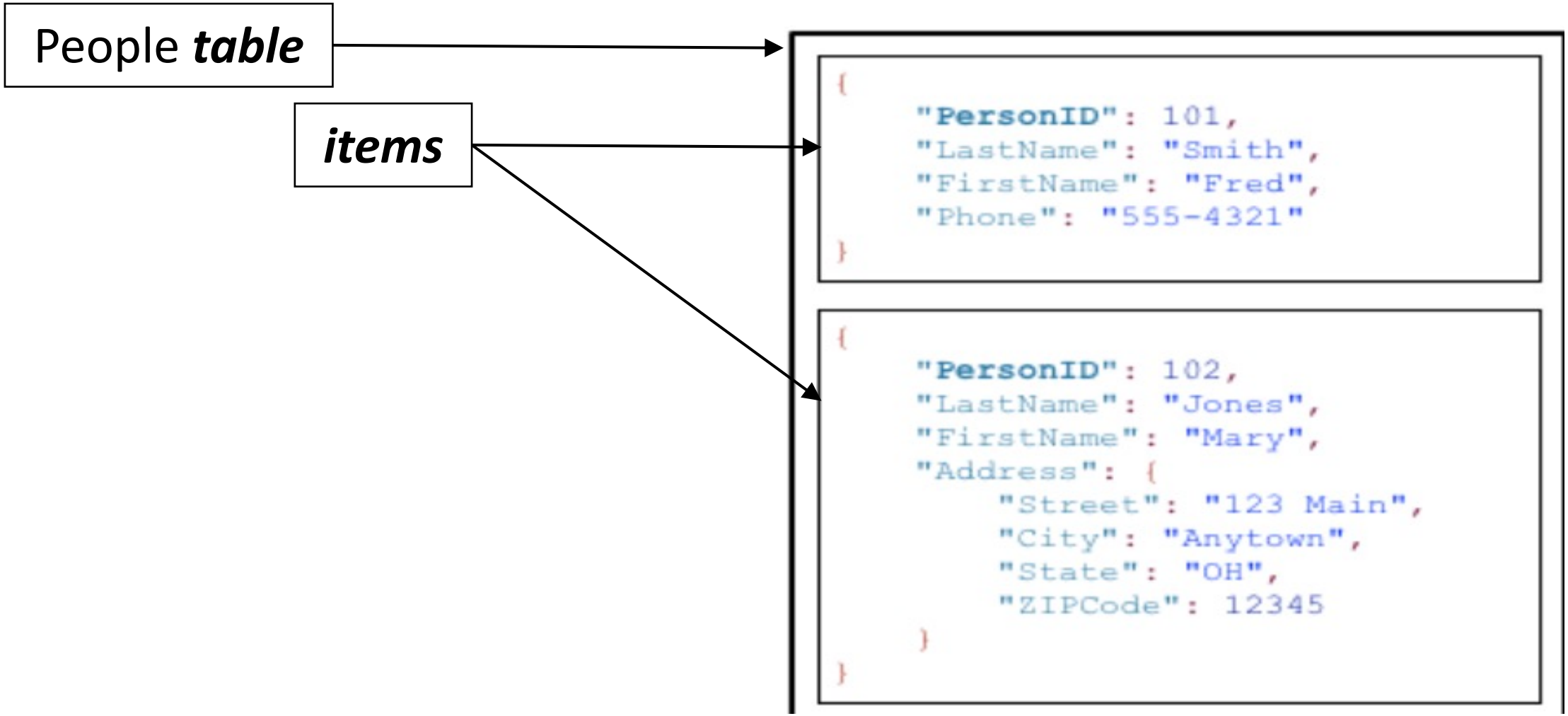
Example: Amazon DynamoDB – Terminology

People ***table***



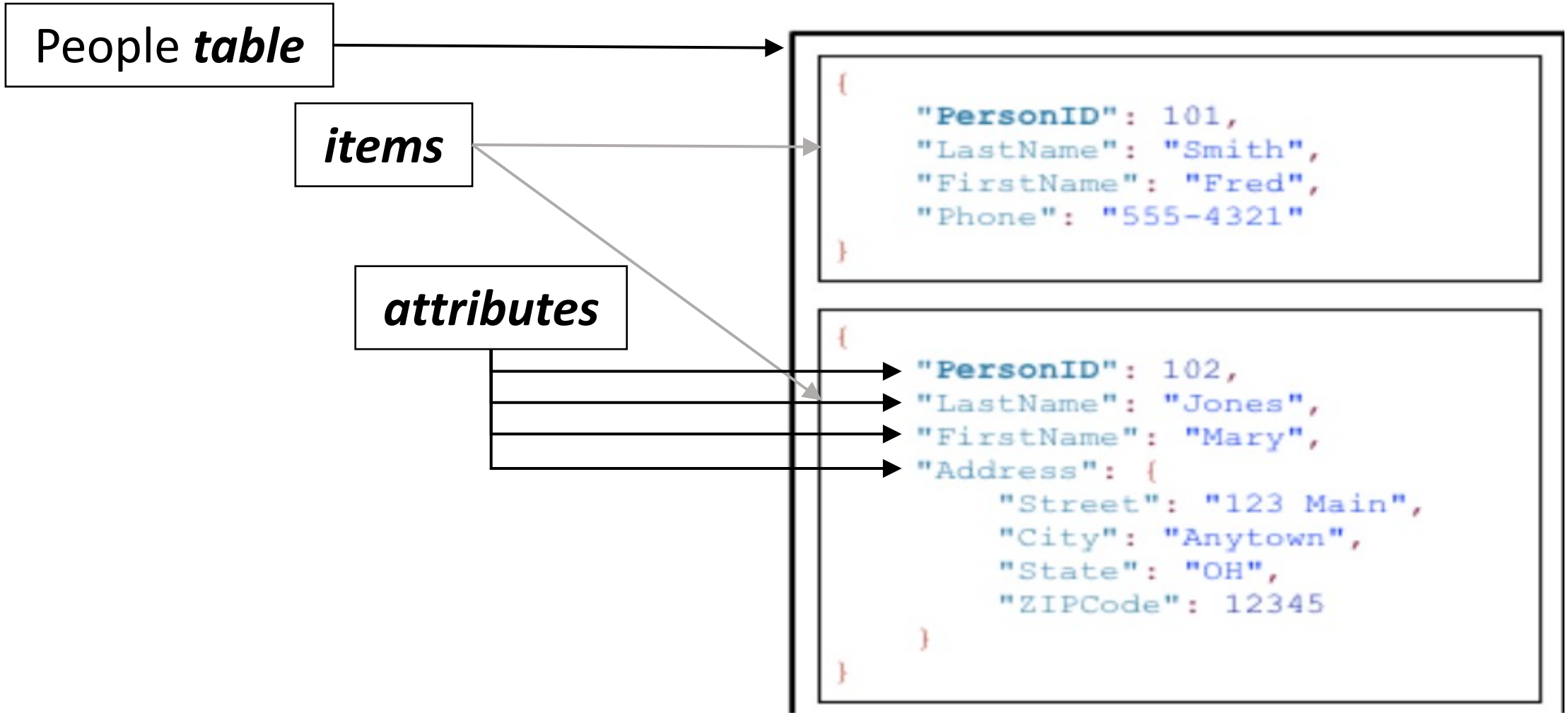
source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.CoreComponents.html

Example: Amazon DynamoDB – Terminology



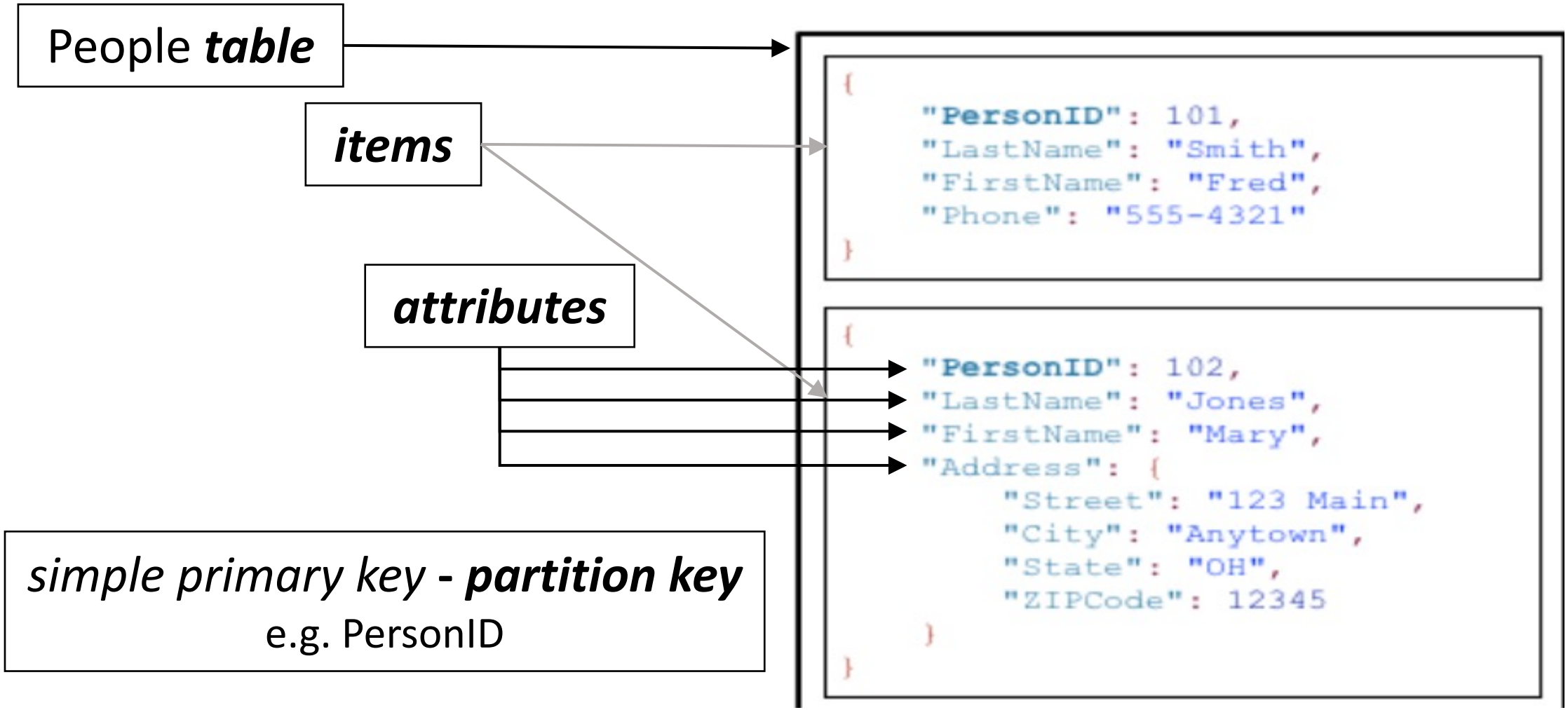
source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.CoreComponents.html

Example: Amazon DynamoDB – Terminology



source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.CoreComponents.html

Example: Amazon DynamoDB – Terminology



source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.CoreComponents.html

Example: Amazon DynamoDB – Terminology

Music ***table***

items

attributes

composite primary key

- **partition key**
  - e.g. Artist
- **sort key**
  - e.g. SongTitle



source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.CoreComponents.html

Partitions and Data Distribution

- Amazon DynamoDB stores data in *partitions*
- A ***partition*** is an allocation of storage for a table
  - backed by solid state drives (SSDs)
  - automatically replicated across multiple Availability Zones within an AWS Region
- DynamoDB manages the partitions

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.Partitions.html

Partitions and Data Distribution

- DynamoDB allocates additional partitions to a table
  - if the table's provisioned throughput is increased, and the existing partitions cannot support the required throughput
  - If more storage space is required

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.Partitions.html

Data Distribution: Partition Key

- For a table with a simple primary key (i.e. partition key only), DynamoDB uses the partition key to determine the partition in which to store/ from where to retrieve the item
  - the value of the partition key is used as an input to an internal hash function, and the output of the hash function (i.e. the hash value of the partition key) determines the corresponding partition

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.Partitions.html

Data Distribution: Partition Key

- For a table with a simple primary key (i.e. partition key only), DynamoDB uses the partition key to determine the partition in which to store/ from where to retrieve the item
  - the value of the partition key is used as an input to an internal hash function, and the output of the hash function (i.e. the hash value of the partition key) determines the corresponding partition
- The partition key of an item is also known as its ***hash attribute***

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.Partitions.html

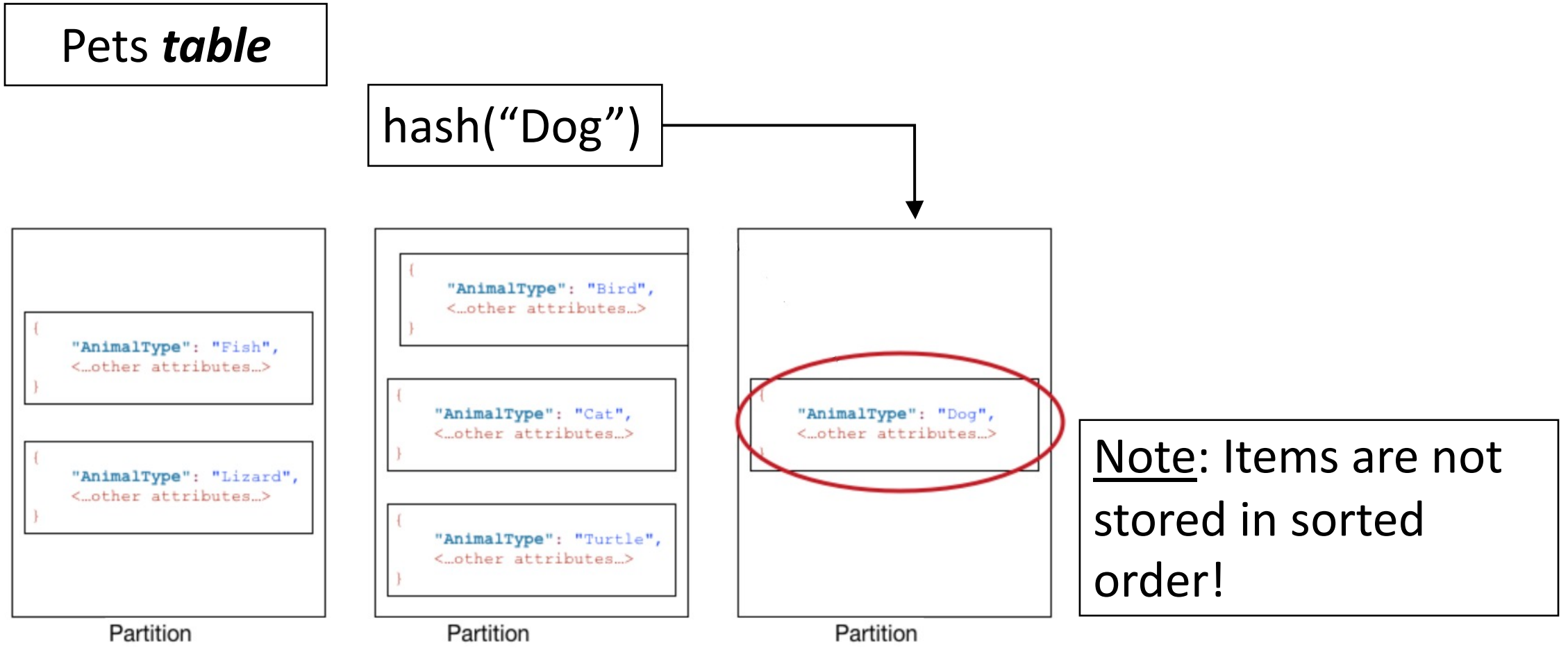
Data Distribution: Partition Key – An Example

Pets ***table***



source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.Partitions.html

Data Distribution: Partition Key – An Example



source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.Partitions.html

Data Distribution: Partition Key & Sort Key

- For a table with a composite primary key (i.e. partition key and sort key), DynamoDB
  - uses the hash value of the partition key to determine the corresponding partition
  - stores the item in ascending order by the sort key

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.Partitions.html

Data Distribution: Partition Key & Sort Key

- For a table with a composite primary key (i.e. partition key and sort key), DynamoDB
  - uses the hash value of the partition key to determine the corresponding partition
  - stores the item in ascending order by the sort key
- The sort key of an item is also known as its ***range attribute***
- There is no limit on the number of different sort key values per partition key value

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.Partitions.html

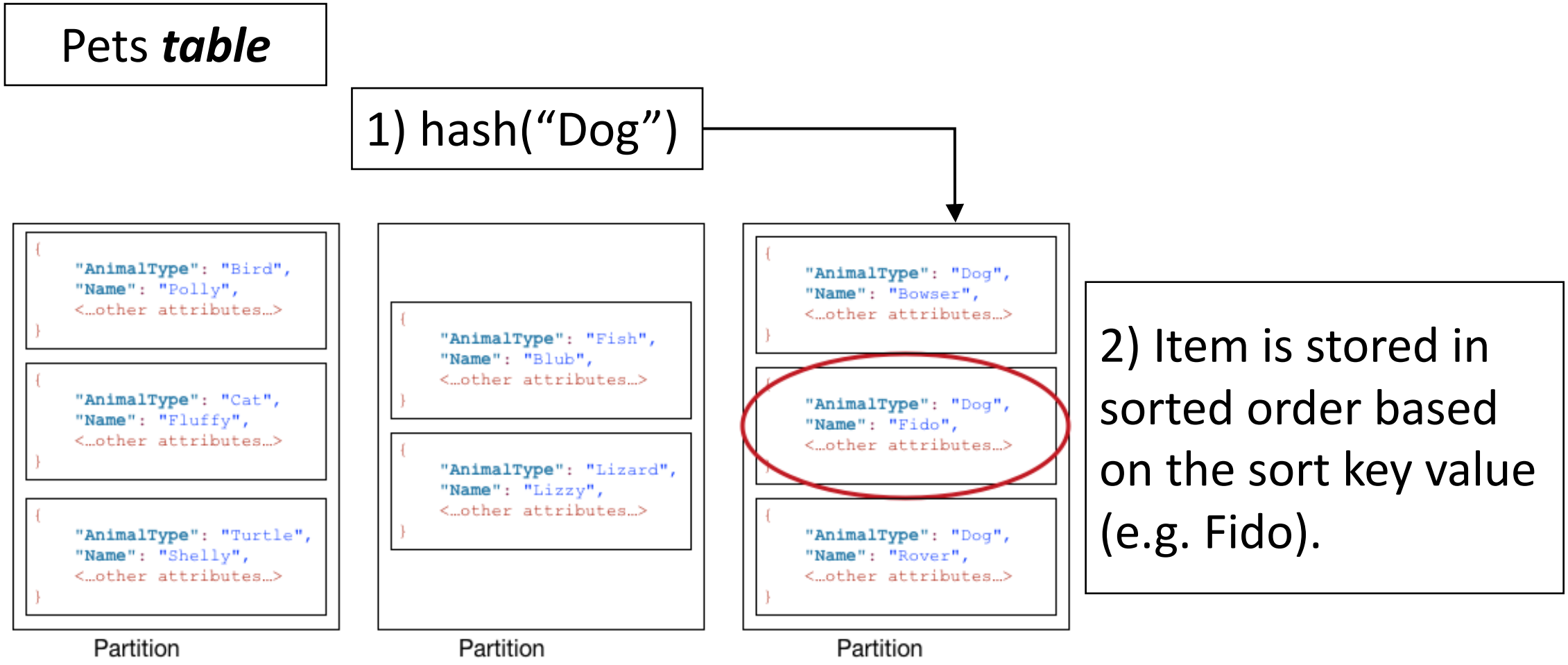
Data Distribution: Partition Key & Sort Key – An Example

Pets ***table***



source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.Partitions.html

Data Distribution: Partition Key & Sort Key – An Example



source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.Partitions.html

Secondary Indexes

- A secondary index enables to query the data in a table using an alternate key
- DynamoDB supports two types of indexes:
  - ***Global secondary index (GSI)*** – an index with a partition key and sort key that can be different from those on the table
  - ***Local secondary index (LSI)*** – an index that has the same partition key as the table, but a different sort key

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.CoreComponents.html

Secondary Indexes

- A secondary index enables to query the data in a table using an alternate key
- DynamoDB supports two types of indexes:
  - ***Global secondary index (GSI)*** – an index with a partition key and sort key that can be different from those on the table
  - ***Local secondary index (LSI)*** – an index that has the same partition key as the table, but a different sort key
- Each table in DynamoDB has a quota of 20 GSIs (default quota) and 5 LSIs per table

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.CoreComponents.html

Secondary Indexes – An Example

Music


```
{
    "Artist": "No One You Know",
    "SongTitle": "My Dog Spot",
    "AlbumTitle": "Hey Now",
    "Price": 1.98,
    "Genre": "Country",
    "CriticRating": 8.4
}
```

```
{
    "Artist": "No One You Know",
    "SongTitle": "Somewhere Down The Road",
    "AlbumTitle": "Somewhat Famous",
    "Genre": "Country",
    "CriticRating": 8.4,
    "Year": 1984
}
```

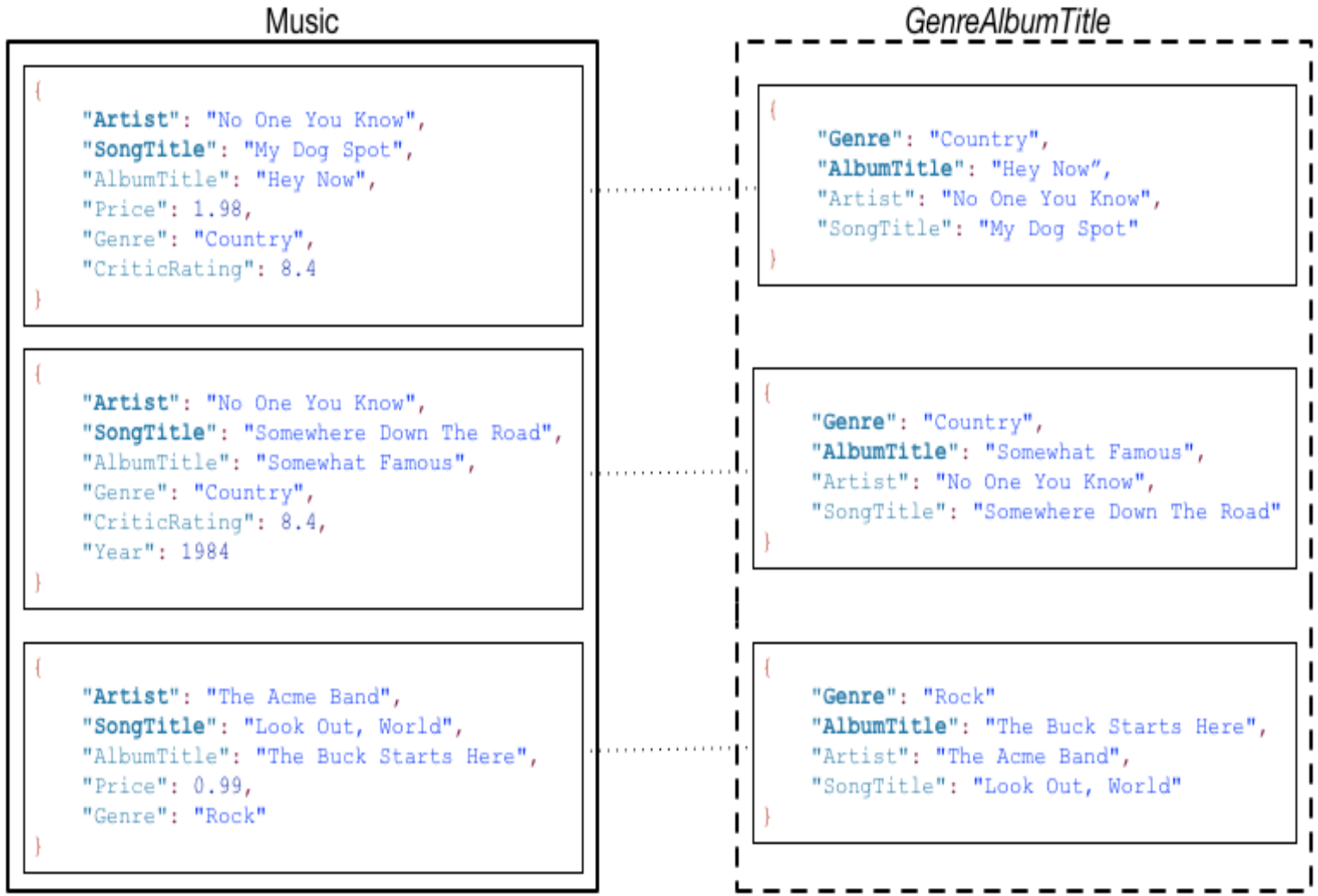
```
{
    "Artist": "The Acme Band",
    "SongTitle": "Look Out, World",
    "AlbumTitle": "The Buck Starts Here",
    "Price": 0.99,
    "Genre": "Rock"
}
```


Query items from Music table by using:

- partition key (e.g. Artist)
- partition key and sort key (e.g. Artist and SongTitle)

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.CoreComponents.html

Secondary Indexes – An Example



Query items from GenreAlbumTitle index by using:

- partition key (e.g. Genre)
- partition key and sort key (e.g. Genre and AlbumTitle)

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.CoreComponents.html

Secondary Indexes

- Every index belongs to a table, i.e. the *base table*
  - e.g. Music is the base table for GenreAlbumTitle index

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.CoreComponents.html

Secondary Indexes

- Every index belongs to a table, i.e. the *base table*
  - e.g. Music is the base table for GenreAlbumTitle index
- DynamoDB maintains indexes automatically
  - if an items is added, updated or deleted in the base table, DynamoDB adds, updates or deletes the corresponding item in any indexes of that table

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.CoreComponents.html

Secondary Indexes

- Every index belongs to a table, i.e. the *base table*
  - e.g. Music is the base table for GenreAlbumTitle index
- DynamoDB maintains indexes automatically
  - if an items is added, updated or deleted in the base table, DynamoDB adds, updates or deletes the corresponding item in any indexes of that table
- When an index is created the attributes to be copied/projected from the base table to the index need to be specified
  - DynamoDB projects, at a minimum, the key attributes from the base table into the index

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.CoreComponents.html

Read Consistency

- Amazon DynamoDB is available in multiple AWS Regions around the world
  - Each Region is independent and isolated from other AWS Regions
- Every AWS Region consists of multiple distinct locations called Availability Zones (AZs)
- Each Availability Zone
  - is isolated from failures in other Availability Zones
  - provides inexpensive, low-latency network connectivity to other Availability Zones in the same Region
- This enables rapid replication of data among multiple AZs in an AWS Region

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadConsistency.html

Read Consistency

- A successful write to the DynamoDB is durable
  - The data is eventually consistent across all storage locations (<= 1 second)

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadConsistency.html

Read Consistency

DynamoDB supports

- **Eventually Consistent Reads** (default)

- **Strongly Consistent Reads**

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadConsistency.html

Read Consistency

DynamoDB supports

- **Eventually Consistent Reads** (default)
  - The response might not show the latest up to date version of a recently completed write operation
    - “dirty reads” i.e. may return stale data
  - If the read request is repeated after a short time, the response should return the latest up to date data
- **Strongly Consistent Reads**

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadConsistency.html

Read Consistency

DynamoDB supports

- **Eventually Consistent Reads** (default)
  - The response might not show the latest up to date version of a recently completed write operation
    - “dirty reads” i.e. may return stale data
  - If the read request is repeated after a short time, the response should return the latest up to date data.
- **Strongly Consistent Reads**
  - The response contains the most up to date data

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadConsistency.html

Read/Write Capacity Mode

- Amazon DynamoDB has two read/write capacity modes
  - On-demand
  - Provisioned (default)
- Controls how you are charged for read and write throughput and how you manage capacity
- ***Throughput*** – is the maximum amount of capacity that an application can consume from a table or index
  - Divided evenly across the partitions

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadWriteCapacityMode.html

Read/Write Capacity Mode – On-Demand

- Flexible billing option capable of serving thousands of requests per second without capacity planning
- Pay-per-request pricing for read and write requests so that you pay only for what you use
- Suitable for
  - New tables with unknown workloads
  - Unpredictable application traffic
  - Paying for only what one uses

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadWriteCapacityMode.html

Read/Write Capacity Mode – Provisioned

- *Read capacity unit*
  - The number of strongly consistent read request per second for an item up to 4 KB in size
  - Eventually consistent reads, one read capacity unit is two reads per second for an item up to 4 KB
- Write capacity unit
  - The number of 1 KB writes per second in size.

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadWriteCapacityMode.html

DynamoDB Streams

- DynamoDB Streams captures data modification events in DynamoDB tables
  - Near real-time, in the order that the events happened
- Each event is stored as a *stream record*
  - A stream record is stored for 24 hours
- Each stream record appears exactly once in the stream

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html

DynamoDB Streams

- If a stream is enabled on a table, DynamoDB Streams captures a *time-ordered* sequence of *item-level* modifications
  - A new item is added to the table
    - The stream records an image of the entire item, including all attributes
  - An item is updated
    - The stream records the "before" and "after" image of any attributes that were updated
  - An item is deleted from the table
    - The stream records an image of the entire item before it was deleted

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html

DynamoDB Streams – Use Cases

- Use DynamoDB Streams together with AWS Lambda to create a trigger
  - Code that runs automatically whenever an event of interest appears in a stream
- DynamoDB Streams enables data replication within and across AWS Regions
- For more uses cases consult: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html

Global Tables

- Provides a fully managed, multi-region, and multi-master database
  - fast, local, read and write performance for massively scaled applications
- Global tables replicate the DynamoDB tables automatically across multiple AWS Regions

source: https://aws.amazon.com/dynamodb/global-tables/

Global Tables

- A ***global table*** is a collection of one or more replica tables
  - 1 replica table / AWS Region
- A ***replica table*** is a single DynamoDB table that is part of a global table
- Each replica has the same table name and primary key schema, and stores the same items

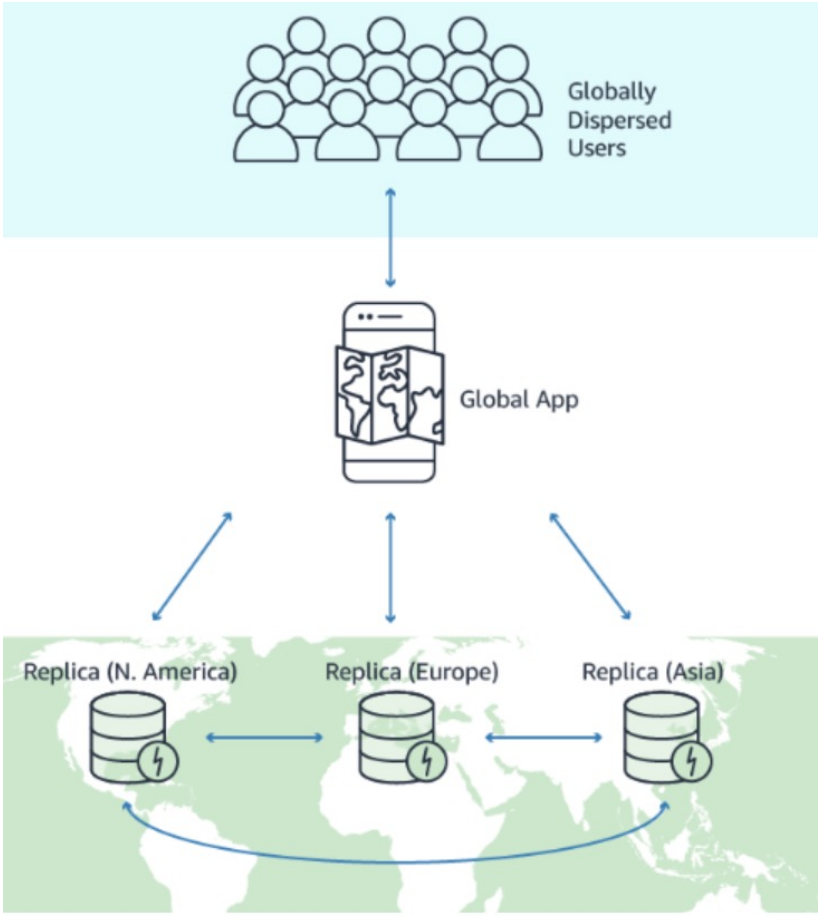


image source: https://aws.amazon.com/dynamodb/global-tables/

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/globaltables_HowItWorks.html

Global Tables

- When a replica table is updated, DynamoDB automatically replicates the changes to the other replica tables in the other AWS Regions

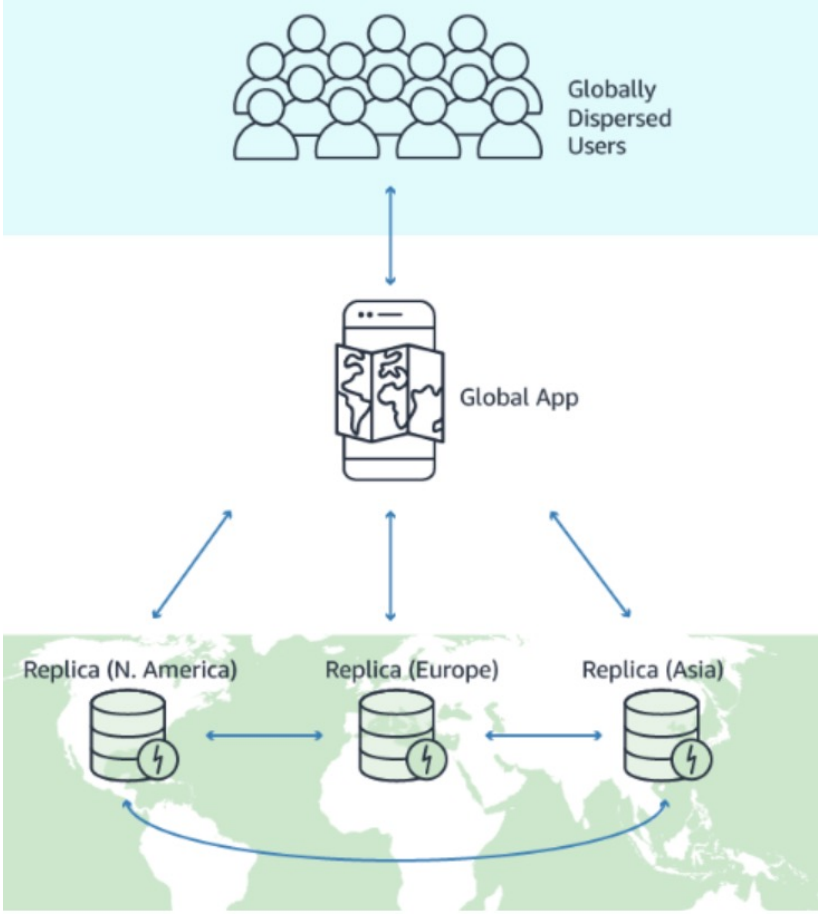


image source: https://aws.amazon.com/dynamodb/global-tables/

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/globaltables_HowItWorks.html

Global Tables

- When a replica table is updated, DynamoDB automatically replicates the changes to the other replica tables in the other AWS Regions
  - Global tables use DynamoDB Streams to replicate changes across replicas

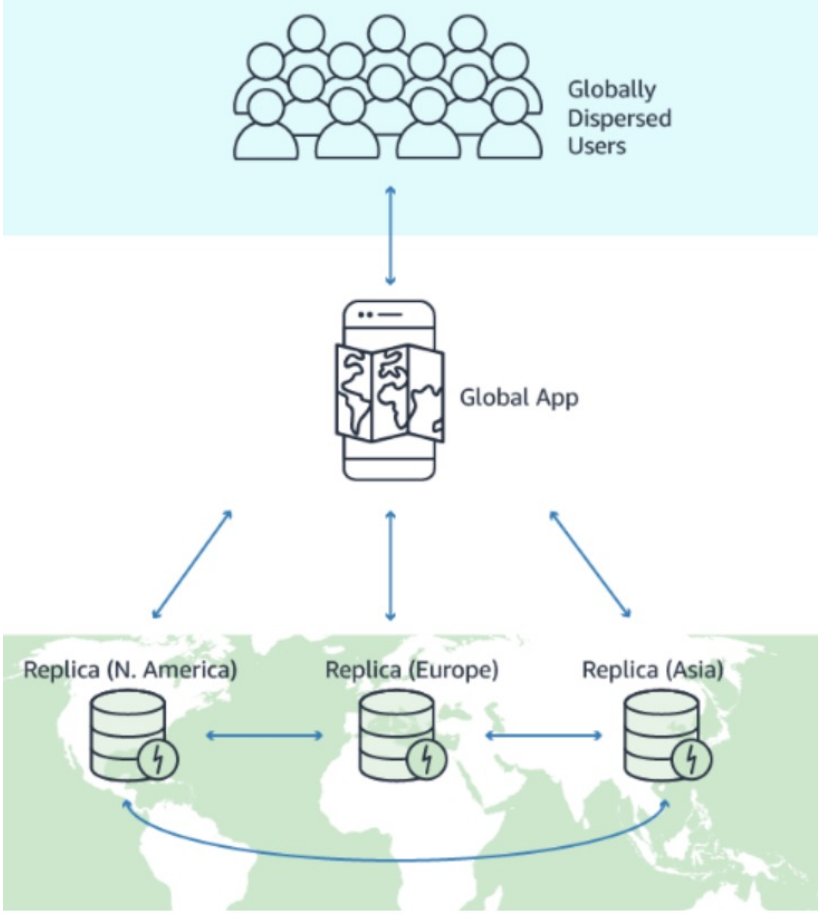


image source: https://aws.amazon.com/dynamodb/global-tables/

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/globaltables_HowItWorks.html

Global Tables

- When any item in a replica table is updated, DynamoDB automatically replicates the changes to the other replica tables in the other AWS Regions
  - Global tables use DynamoDB Streams to replicate changes across replicas
  - Any update made to an item in a replica table is captured in a stream record, and the change is replicated across all the other replica tables

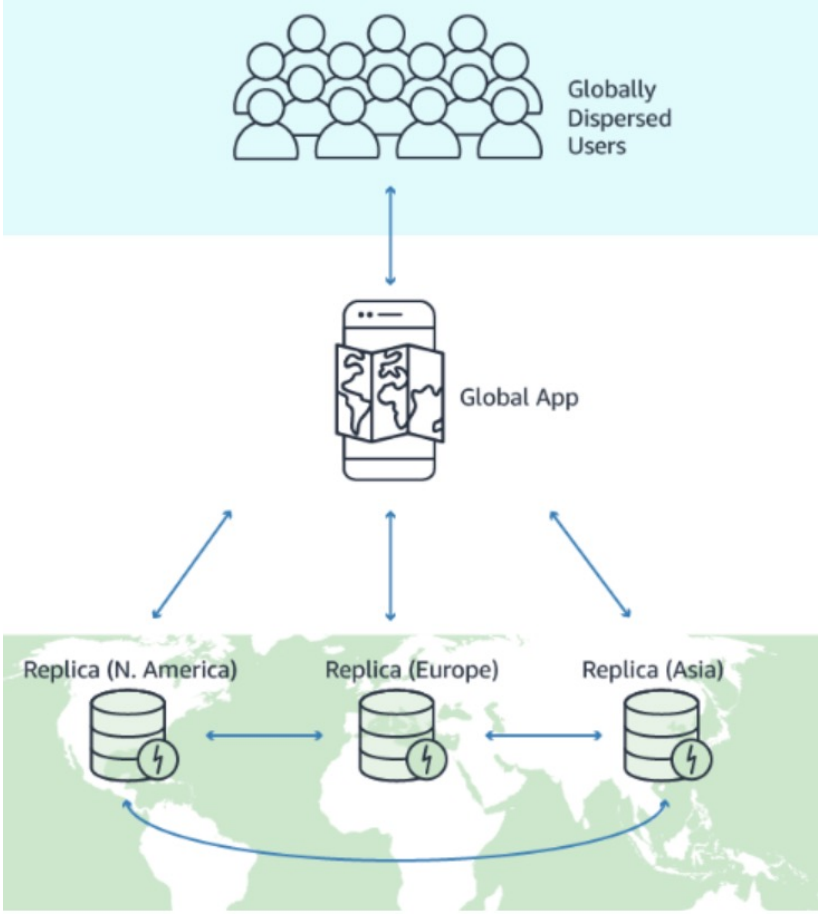
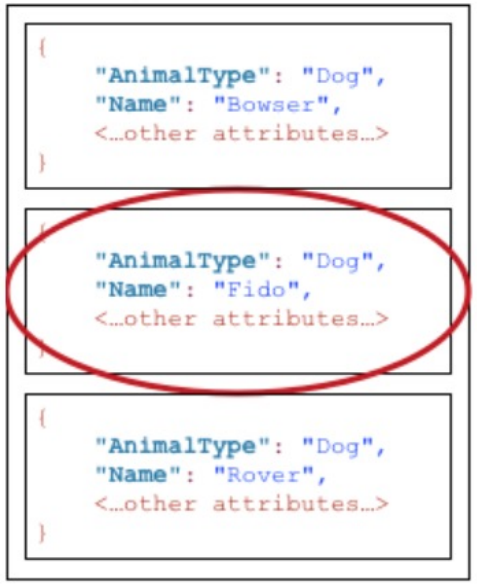


image source: https://aws.amazon.com/dynamodb/global-tables/

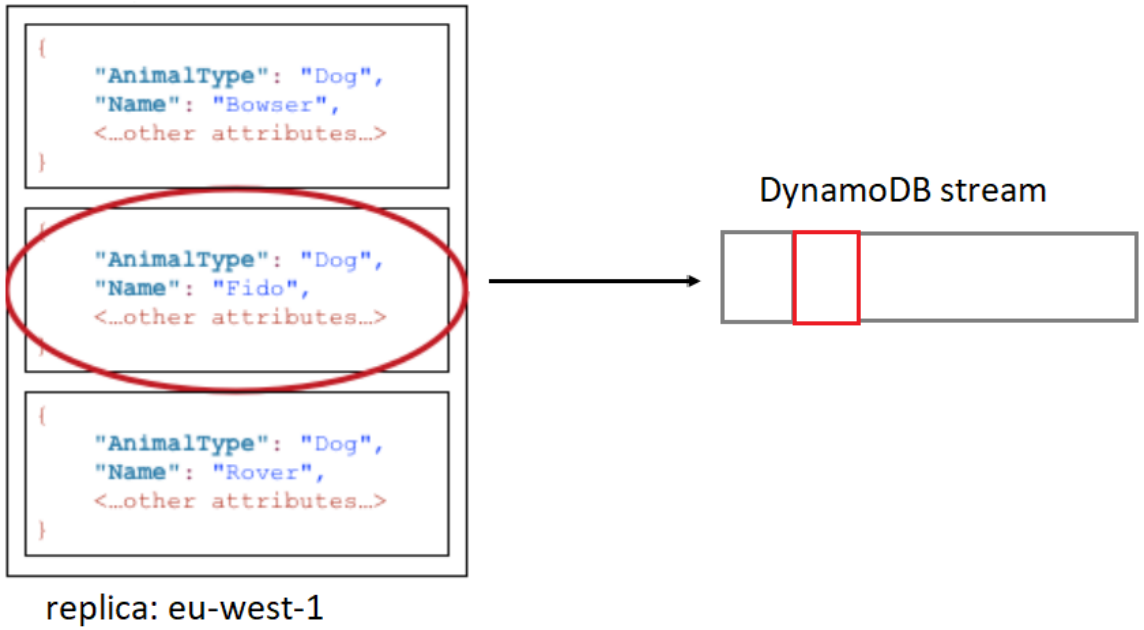
source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/globaltables_HowItWorks.html

Global Tables – An Example

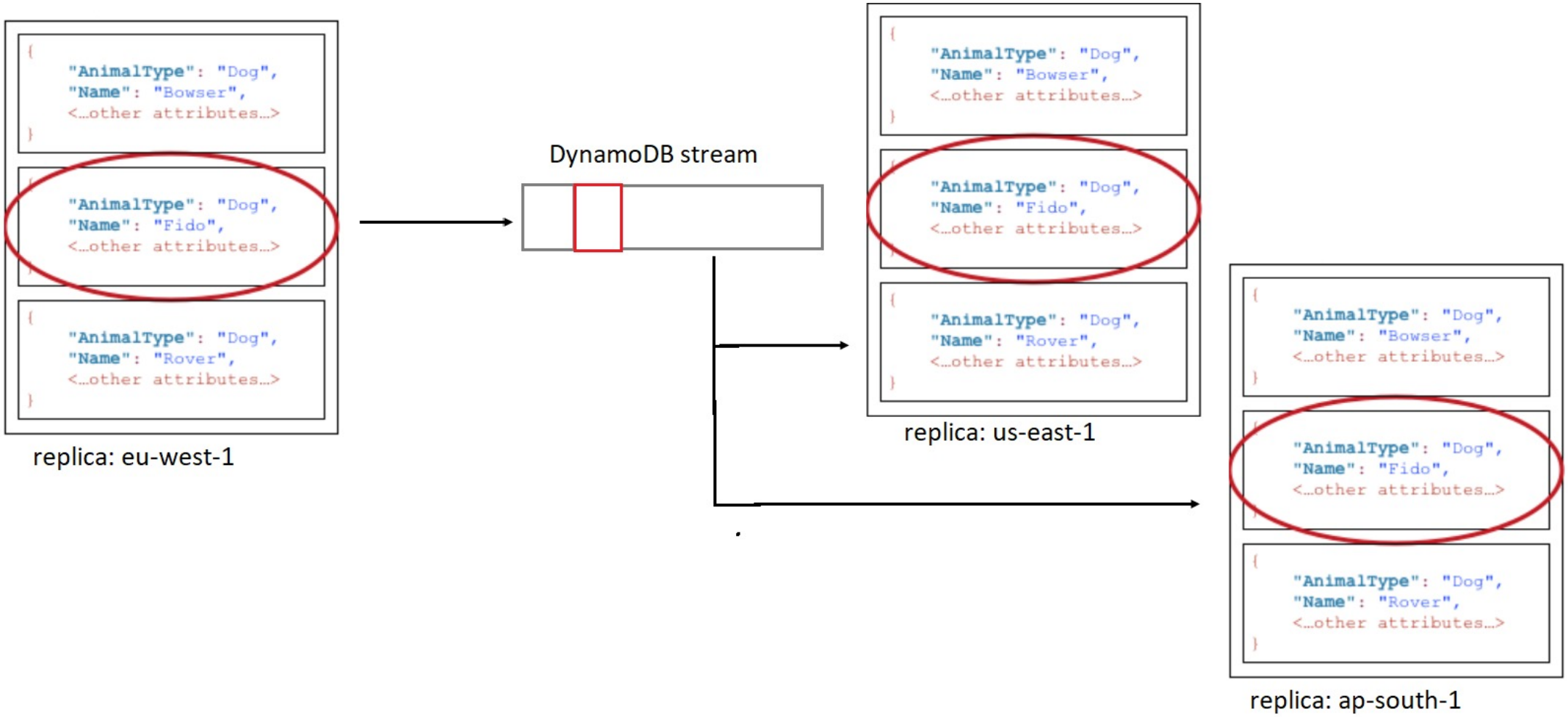


replica: eu-west-1

Global Tables – An Example (contd.)



Global Tables – An Example (contd.)



Global Tables – Read Consistency

- In a global table, a newly written item is replicated to all the replica tables within seconds

- DynamoDB does not support partial replication (i.e. of only some of items)

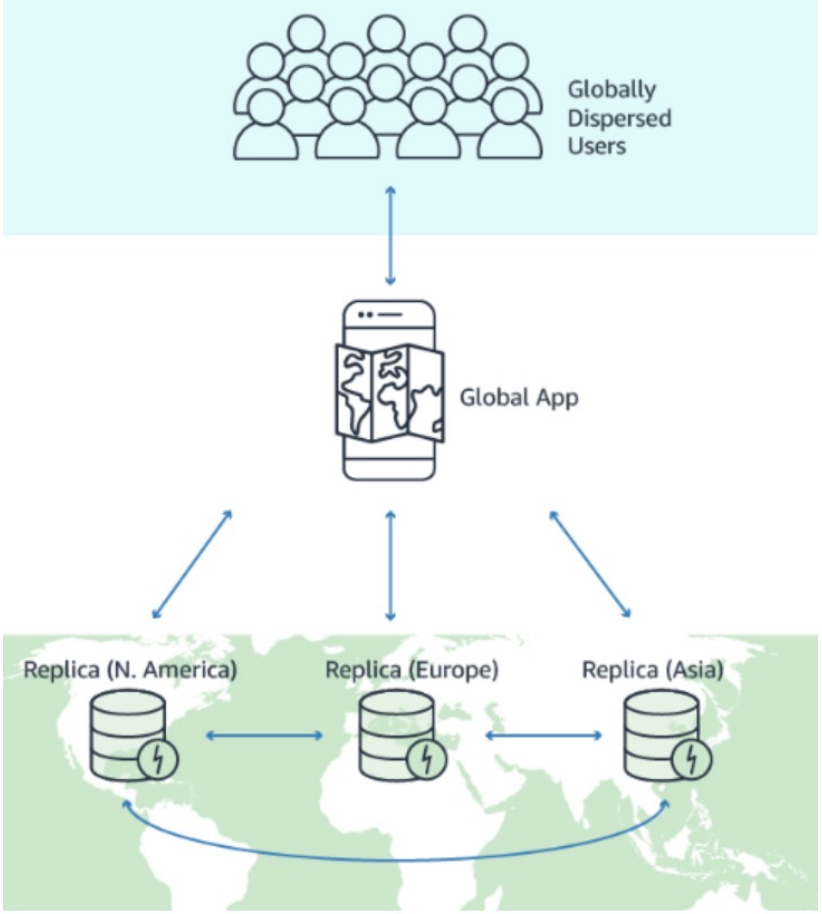


image source: https://aws.amazon.com/dynamodb/global-tables/

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/globaltables_HowItWorks.html

Global Tables – Read Consistency

- An application can read and write data to any replica table

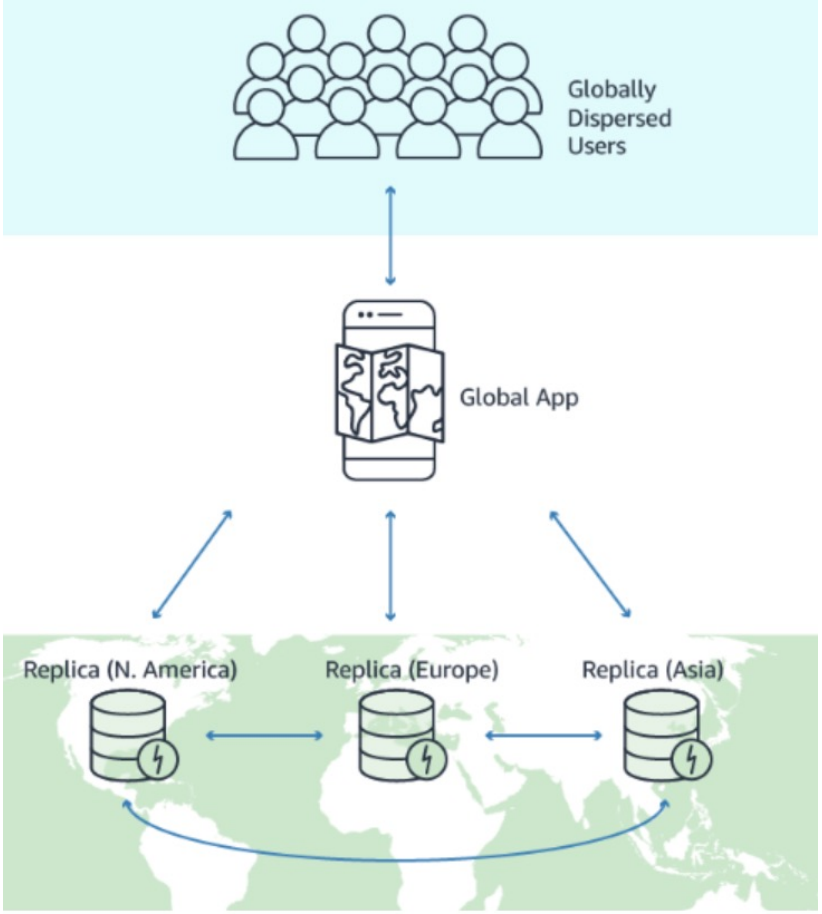


image source: https://aws.amazon.com/dynamodb/global-tables/

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/globaltables_HowItWorks.html

Global Tables – Read Consistency

- An application can read and write data to any replica table
- If application only uses eventually consistent reads and only reads data from one AWS Region no changes are required

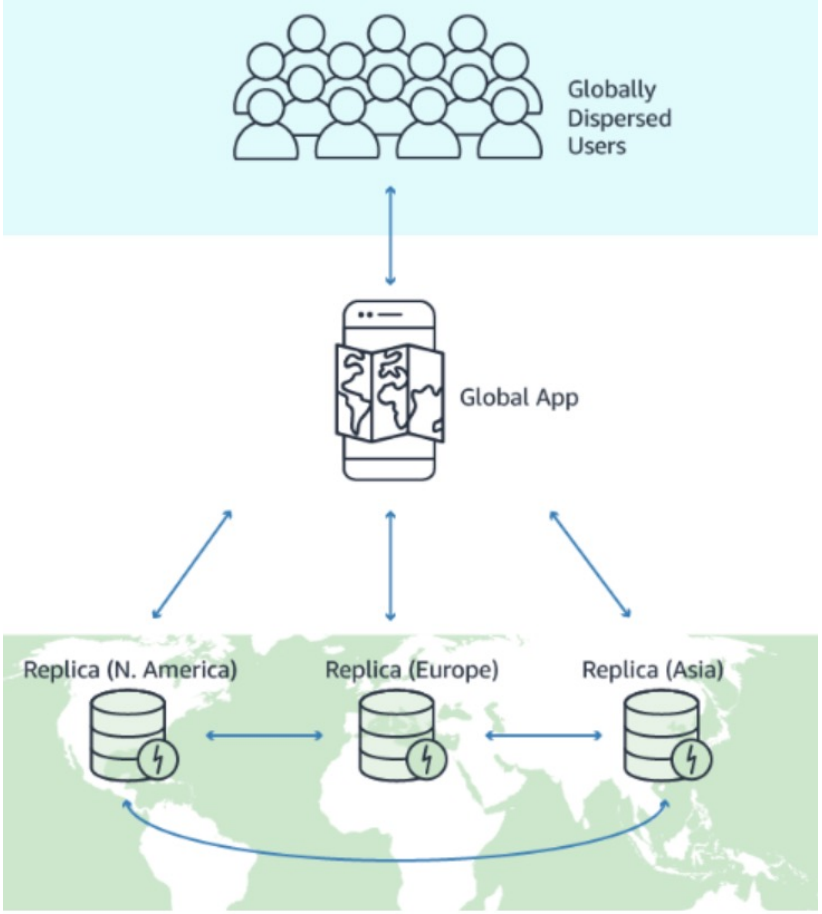


image source: https://aws.amazon.com/dynamodb/global-tables/

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/globaltables_HowItWorks.html

Global Tables – Read Consistency

- An application can read and write data to any replica table
- If application only uses eventually consistent reads and only reads data from one AWS Region no changes are required
- If application requires strongly consistent reads, it must perform all of its strongly consistent reads and writes in the same AWS Region

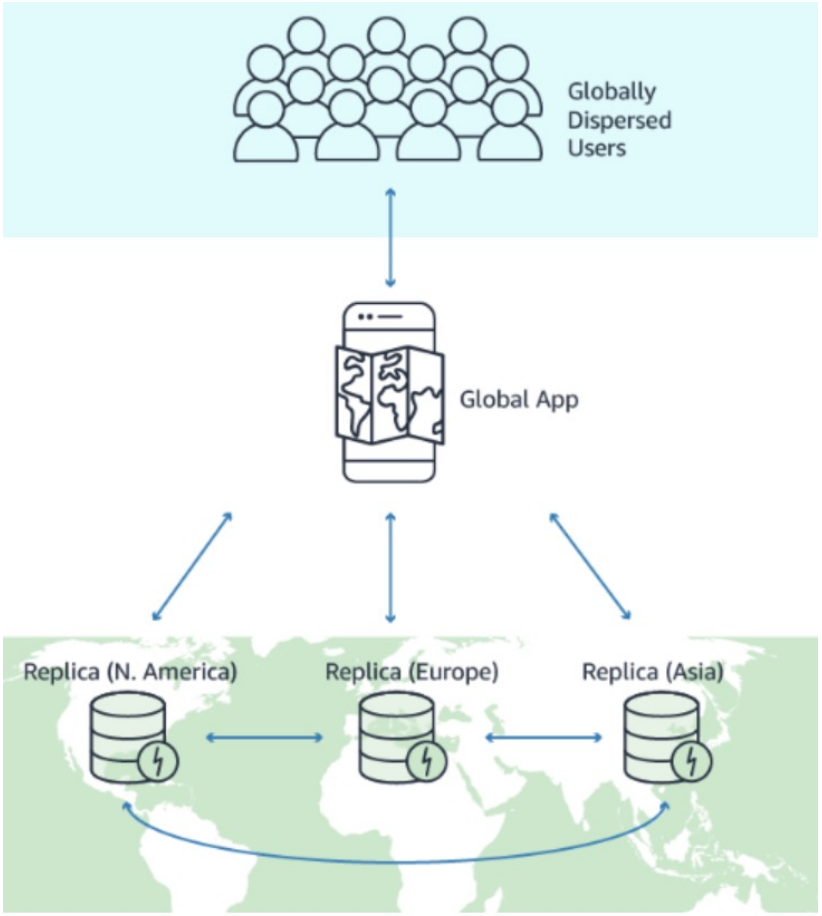


image source: https://aws.amazon.com/dynamodb/global-tables/

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/globaltables_HowItWorks.html

Global Tables – Read Consistency

- An application can read and write data to any replica table
- If application only uses eventually consistent reads and only reads data from one AWS Region no changes are required
- If application requires strongly consistent reads, it must perform all of its strongly consistent reads and writes in the same AWS Region
- DynamoDB does not support strongly consistent reads across Regions

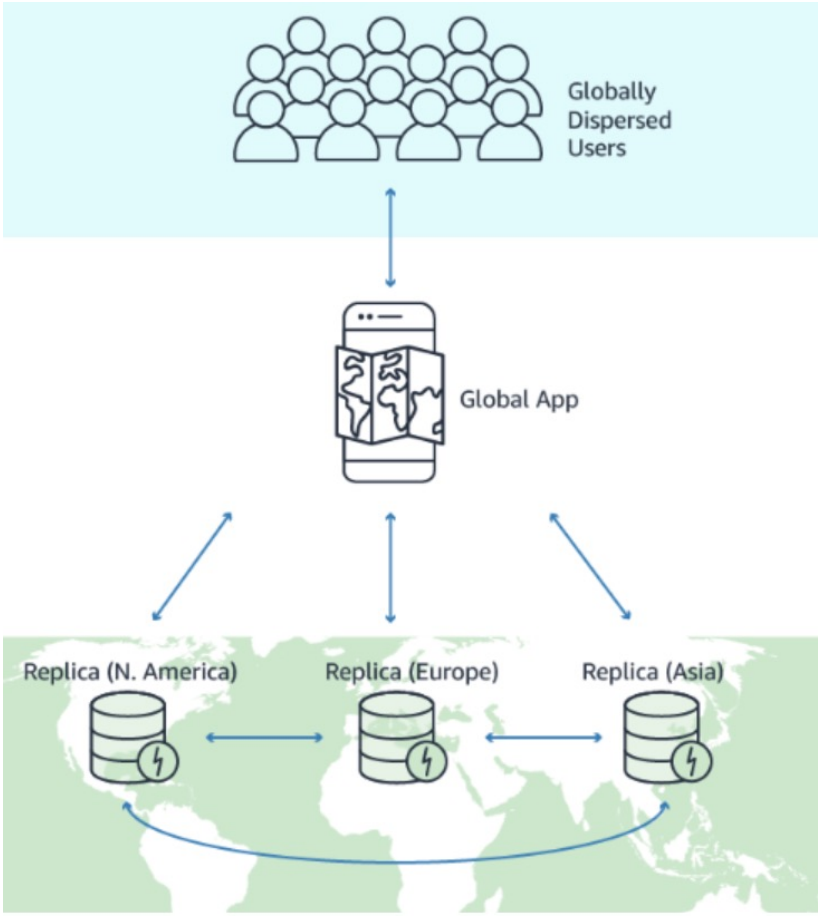


image source: https://aws.amazon.com/dynamodb/global-tables/

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/globaltables_HowItWorks.html

Global Tables – Conflict Resolution

- When applications update the same item in different AWS Regions at similar time, conflicts can appear
- Eventual Consistency
  - global tables use a last-writer-wins reconciliation for concurrent updates (i.e. latest update)

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/globaltables_HowItWorks.html

Global Tables – Benefits

- Multi-master replication ensure data accessed by globally distributed applications are up to date
- Performance
  - Global tables enables to read and write data locally
- Easy to setup and work with
- Availability, durability and multi-region fault tolerance
- Consistency and conflict resolution

source: https://aws.amazon.com/dynamodb/global-tables/

Working with Amazon DynamoDB

- AWS Management Console
- AWS CLI
- REST API
- AWS SDKs

Amazon DynamoDB Core API Operations

- Control Plane
- Data Plane
- DynamoDB Streams
- Transactions

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.API.html/

Amazon DynamoDB Core API Operations

- Control Plane
  - Create and manage DynamoDB tables
- Data Plane

- DynamoDB Streams

- Transactions

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.API.html/

Amazon DynamoDB Core API Operations

- Control Plane
  - Create and manage DynamoDB tables
- Data Plane
  - Create, read, update, and delete (CRUD) actions on data in a table
- DynamoDB Streams

- Transactions

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.API.html/

Amazon DynamoDB Core API Operations

- Control Plane
  - Create and manage DynamoDB tables
- Data Plane
  - Create, read, update, and delete (CRUD) actions on data in a table
- DynamoDB Streams
  - Enable or disable a stream on a table, and allow access to the data modification records contained in a stream
- Transactions

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.API.html/

Amazon DynamoDB Core API Operations

- Control Plane
  - Create and manage DynamoDB tables
- Data Plane
  - Create, read, update, and delete (CRUD) actions on data in a table
- DynamoDB Streams
  - Enable or disable a stream on a table, and allow access to the data modification records contained in a stream
- Transactions
  - Provides atomicity, consistency, isolation, and durability (ACID) enabling to maintain data correctness in applications more easily

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.API.html/

API Operations – Create a Table

- CreateTable - an asynchronous operation
  - After the table is created, DynamoDB sets the TableStatus to ACTIVE
  - Can perform read and write operations only on an ACTIVE table

source: https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_Operations_Amazon_DynamoDB.html

API Operations – Create an Item

- PutItem
  - Creates a new item, or replaces an old item with a new item
  - Can perform a conditional put operation
    - Add a new item if the one with the specified primary key doesn't exist
    - Replace an existing item if it has certain attribute values

source: https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_Operations_Amazon_DynamoDB.html

API Operations – Update an Item

- UpdateItem
  - Edits an existing item's attributes, or adds a new item to the table if it does not already exist
  - Can put, delete, or add attribute values
  - Can perform a conditional update on an existing item
    - Insert a new attribute name-value pair if it doesn't exist
    - Replace an existing name-value pair if it has certain expected attribute values

source: https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_Operations_Amazon_DynamoDB.html

API Operations – Update an Item

```
table.update_item(
    Key = {
        ‘pps': ‘P12345678M'
    },
    UpdateExpression = 'SET salary = salary + :val1',
    ExpressionAttributeValues = {
        ':val1': 1000
    }
)
```

source: https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/dynamodb.html#DynamoDB.Clier

API Operations – Delete an Item

- DeleteItem
  - Deletes a single item in a table by primary key
  - Can perform a conditional delete operation
    - Deletes the item if it exists
    - Deletes the item if it has an expected attribute value

source: https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_Operations_Amazon_DynamoDB.html

API Operations – Condition expressions

- PutItem, UpdateItem and DeleteItem operations allows to specify a *condition expression* to determine the items that should be modified

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Expressions.ConditionExpressions.html

API Operations – Condition expressions

- PutItem, UpdateItem and DeleteItem operations allows to specify a *condition expression* to determine the items that should be modified
  - e.g.

    UpdateItem Operation

    Primary key: orderId = 1

    Set: year = 2020

    ConditionExpression:

    year < 2020

source: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Expressions.ConditionExpressions.html

API Operations – Read an Item

- GetItem
  - Returns a set of attributes for the item with the given primary key
  - If there is no matching item, GetItem does not return any data and there will be no Item element in the response

source: https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_Operations_Amazon_DynamoDB.html

API Operations – Query a Table

- Query
  - Finds items based on primary key values.
  - Can query any table or secondary index that has a composite primary key (a partition key and a sort key)

source: https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_Operations_Amazon_DynamoDB.html

API Operations – Query a Table

- Query
  - Finds items based on primary key values.
  - Can query any table or secondary index that has a composite primary key (a partition key and a sort key)

```
response = table.query (
    KeyConditionExpression = Key(‘pps').eq(‘P12345678M ')
)
```

source: https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_Operations_Amazon_DynamoDB.html

API Operations – Scan a Table

- Scan
  - Returns one or more items and item attributes by accessing every item in a table or a secondary index
  - The result set can be refined by using a filter expression.

source: https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_Operations_Amazon_DynamoDB.html

API Operations – Scan a Table

- Scan
  - Returns one or more items and item attributes by accessing every item in a table or a secondary index
  - The result set can be refined by using a filter expression.

```
response = table.scan (
    FilterExpression = Attr(‘salary').lt(40000)
)
```

source: https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_Operations_Amazon_DynamoDB.html

Resources

- Thomas Connolly, Carolyn Begg 2014, Database Systems: A Practical Approach to Design, Implementation, and Management, 6th Edition Ed., Pearson Education [ISBN: 1292061189]
- Amazon DynamoDB: https://aws.amazon.com/dynamodb/
- Programming with DynamoDB and AWS SDKs: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Programming.html
- Amazon DynamoDB Developer Guide: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html

Resources

- DynamoDB Streams: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html
- AWS Regions in which DynamoDB is available https://docs.aws.amazon.com/general/latest/gr/rande.html#ddb_region
- Amazon DynamoDB API Reference https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/Welcome.html